MACHINE PROBLEM 1

Title: String Manipulation Toolkit

Overview:
Create a Java program that serves as a toolkit for string manipulation. The toolkit should include a set of functions that perform various operations on strings. Each method should focus on a specific aspect of string manipulation, utilizing relevant Java string methods.

Development Requirements:
You are tasked with creating a text analysis tool in Java. The tool should perform various analyses on a given text and provide relevant information. Implement the following functionalities:

1. **String Reversal:** Implement a method that reverses a given string.
2. **Palindrome Checker:** Implement a method that checks whether a given string is a palindrome.
3. **Anagram Checker:** Implement a function that checks if two strings are anagrams.
4. **Word Count:** Implement a function that counts the number of words in a given string.
5. **Character Count:** Implement a method that counts the total number of characters in the text (excluding spaces).
6. **Substring Finder:** Implement a method that finds all occurrences of a specified substring in a given string.
7. **Lower Case Converter:** Implement a method that converts a given string to lowercase.
8. **Upper Case Converter:** Implement a method that converts a given string to uppercase.
9. **Vowel Remover:** Implement a method that removes all vowels from a given string.
10. **Consonant Remover:** Implement a function that removes all consonants from a given string.

The program should allow the user to choose a Sting Manipulation Tool in the given options in the screen. The user should only input 1-10 to choose on any of the option. Any numbers aside from 1-10, should not be accepted by the program. Once the user already pick a number, the program will ask the user again to enter the string consisting of 3 sentences and this should be processed depending on the number picked by the user.

Process Flow Diagram:

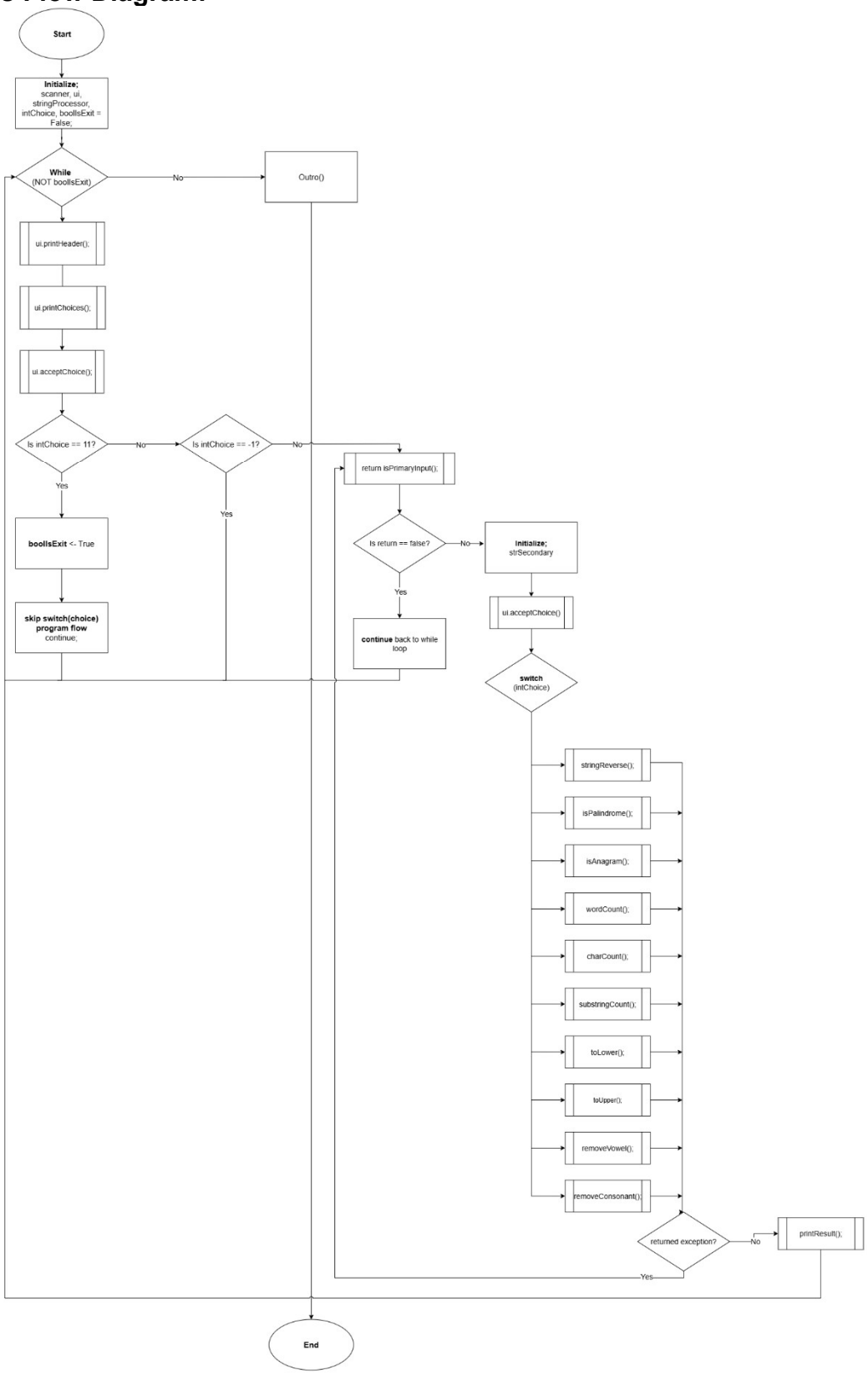


Fig 1. Basic Process Flowchart Diagram

Variable Definition:

- **strInput -** container for the input string to validate
- **strInput** – private field of the StringMethods object used to store the string to manipulate via its methods
- **arrayOriginal** – an unaltered character-array container used to store a converted string, relative to stringReverse()
- **arrayReversed** – the reversed counterpart of the character array *arrayOriginal*, can be found and processed by stringReverse();
- **stringReversed** – used to store the return value of stringReverse(), the string-converted from the character array *arrayReversed()*
- **strToCompare** – an argument-variable for secondary string, besides the primary string(strInput) to facilitate comparison for methods that require reference/comparison to another string.
- **arrInput** – character-array container used to store the input converted string
- **arrToCompare** – character-array counterpart, converted from *strToCompare* for methods that require it.
- **mapFreq** – a hashmap/map for frequency of elements, can be found when processing *strInput* and *strToCompare* to record occurrences of unique elements
- **character** – used as a for-loop character buffer for the enhanced (and normal)java for loop concerning arrays
- **intFreq** – used to store integer key values extracted from *mapFreq,* to determine if there is a character that has never been recorded before.
- **arrText** – character-array container used to store the cleaned(lowercased, trimmed) and converted *strInput*
- **intSubstring** – used to store count of how many word-characters were last seen since starting to account for a new word. For example (string: hello, intSubstring = 5)
- **intWordCount** – used to store count of all the valid word-substrings counted, which is equivalent to words, thus word count.
- **charPrevious** – used to store the last seen character while traversing the character-array**.** Keeps record despite resetting the substring counting process.
- **strNoSpace** – a container for the altered version of the *strInput,* where all whitespaces are removed, found and processed by charCount
- **index** – stores the index of the first occurrence of the specified substring using .indexOf, serves as a double-purpose, that identifies how far in the length of *strInput* the program has processed in substringCount()
- **i** – the quick and universal way to name the iteration of a for loop, used in this project likewise
- **ISVOWEL** – a variable of Pattern type, used to hold the compiled version of a the specified regex pattern that pertains to all vowels in the alphabet. Used under removeConsonants()
- **ISCONSONANT** – a variable of Pattern type, used to hold the compiled version of a the specified regex pattern that pertains to all consonants in the alphabet. Used under removeVowel()

- **arrMenu** – a 2D String array used to contain the method names, and respective descriptions for all methods in the StringMethods class
- **strTemp** – used to store strings temporarily, returned as value after one or two operations(ex. reading a string input).
- **intMaxChoice** – the defined maximum number of choices, which defines validity of input when using acceptChoice() method.
- **intChoice –** a storage for an integer resembling a choice from the displayed choices. Found in various code blocks where it requires users to choose from a menu of choices.
- **e** – temporary container for Exception types. Used to either display or hold them in try-catch blocks throughout the program
- **strMethodName** – serves as a string buffer for the method names under *arrMenu* for printing
- **strDescription** – serves as a string buffer for the respective descriptions under *arrMenu* for printing
- **intMinSentence** – defines the specified minimum amount of sentences(3 based on project instructions) that serves to validate primary string inputs when using StringMethods methods
- **intCount** – used as a general container for counting occurrences/frequencies, throughout the program.
- **intMethodName** – used specify an integer-equivalent representation of a method under the valid StringMethods method choices, program defined.
- **ui** – used as a variable for the instantiated object of the UI class, which holds all printing and user input interfaces, program defined.
- **scanner** – used a variable for the instantiated object of the Scanner class, which is used universally in the program to read input from system.in
- **stringProcessor** – used a variable for the instantiated object of the StringMethods class, which holds all string manipulation toolkit methods
- **strResult** – stores the result of string manipulation methods, to later print in the printResults() method
- **strSecondary** – another, more formally named container for the secondary string used in certain StringMethods methods that require it, can be found in. Found and processed in various parts of Main class
- **boolIsExit** – a Boolean variable used to hold the state of the main while loop in the main program flow.
- **intTemp** – used to hold a temporary integer return value, in main, it is used to hold substringCount return value for printing
- **boolTemp** – used to hold a temporary boolean return value, in main, it is used to hold the return value of isAnagram for printing

Method Definition:

- **isEmpty (static)**
  Return type - void
  Parameters - String *strInput*
  Description - Checks if strInput.isEmpty is true, then throws IllegalArgumentException for callers to catch
  Code Snippet:

```
public static void isEmpty(String strInput) { if (strInput.isEmpty()) throw new IllegalArgumentException(s: "Input string cannot be empty."); }
```

- **clean (static)**
  Return Type - String
  Parameter/s – String *strInput*
  Description – “Cleans” by removing all non-alphabetical characters in a string and putting all alphabetical characters in lowercase
  Code Snippet:

```
public static String clean(String strInput) {
    return strInput.toLowerCase()
            .replaceAll(regex: "[^a-z]", replacement: "");
}
```

- **rmSpace (static)**
  Return Type - String
  Parameter/s – String *strInput*
  Description – removes all occurrences of a whitespace character in a string
  Code Snippet:

```
public static String rmSpace(String strInput) {
    return strInput.replaceAll(regex: "\\s+", replacement: "");
}
```

- **setString**
  Return Type - void
  Parameter/s – String *strInput*
  Description – a public setter of class StringMethods, accesses and sets the strInput field of the class
  Code Snippet:

```
public void setString(String strInput){
    if (strInput == null) strInput = "";
    this.strInput = strInput;
}
```

- **getString**
  Return Type - String
  Parameter/s – none
  Description – a public getter of class StringMethods, accesses and returns the strInput field of the class
  Code Snippet:

```
public String getString(){
    return strInput;
}
```

- **stringReverse**
  Return Type - String
  Parameter/s – none
  Description – using the *strInput* field of the class, reverses the order of all characters in the said String by convertion to a character-array and for-loop until n-length of the original string. It then converts the character-array back into a string and returns it.
  Code Snippet:

```java
public String stringReverse() {
    Utils.isEmpty(strInput);

    char[] arrayOriginal = strInput.toCharArray(), arrayReversed = new char[arrayOriginal.length]; // create a new character array in the size of input

    for(int i = 0; i < arrayOriginal.length; i++) {
        arrayReversed[i] = arrayOriginal[arrayOriginal.length - 1 - i];
    }

    String stringReversed = String.copyValueOf(arrayReversed);
    return stringReversed;
}
```

- **isPalindrome**
  Return Type - boolean
  Parameter/s – none
  Description – using the *strInput* field of the class, it reverses the input string, and uses this to see if the reversed string is equal to the original string. It uses equalsIgnoreCase() to check. It then returns the boolean result.
  Code Snippet:

```java
public boolean isPalindrome() {
    Utils.isEmpty(strInput);
    String strTemp = Utils.clean(this.stringReverse());
    return (Utils.clean(strInput)).equalsIgnoreCase(strTemp);
}
```

- **isAnagram**
  Return Type - boolean
  Parameter/s – String *strToCompare*
  Description – using the *strInput* field of the class and the argument *strToCompare,* compares the two strings by recording and counting frequency of all unique characters from *strInput,* and then by subtracting frequency of all unique characters from *strToCompare.* This ensures that all that is recorded is then taken away exactly, which is proof that they are anagrams of each other and returns true, otherwise it returns false.

Code Snippet:

```
Map<Character, Integer> mapFreq = new HashMap<>();

for (char character: arrInput) {
    mapFreq.put(character, mapFreq.getOrDefault(character, defaultValue: 0) + 1);
}

for (char b: arrToCompare) {
    int intFreq = mapFreq.getOrDefault(b, defaultValue: 0);  // if b is recorded for the first time, it returns default val 0
    if (intFreq == 0) return false;                          // if a key is recorded for the first time, then return false.
    mapFreq.put(b, intFreq - 1);
}

for (Integer index : mapFreq.values()) {                     // check if all elements that were counted all became 0 after subtraction loop
if (index!= 0) return false;
}
return true;
```

- **wordCount**
  Return Type - int
  Parameter/s – none
  Description – using the *strInput* field of the class, we determine word counts by setting rules on what a word is; particularly it should consist of alphabetical characters including “ ’ ”, and “-” for some words, and for those special characters, they cannot be back-to-back(e.g ‘’ or --). Special words are also considered like “I” and “A” that words allow to be composed of. The variable *strInput* is converted to a character-array and each index is determines count based off these rules. For each valid character a new substring count is started, that increments *intSubstring,* and when a word-terminating character is reached (e.g not an alphabet), it is considered the end of a word, hence the *intWordCount* is incremented. It then returns the wordCount after successfully going through n length = *strInput* length
  Code Snippet:

```
for(char a: arrText) {

    if (!isAllowed(a)) {                                                    // if its not allowed, set substring to next character immediately
        if (intSubstring > 1 || isSpecialWord(charPrevious)) intWordCount++;        // +1 in word count if substring length reached
        intSubstring = 0;
        charPrevious = a;                                                       // either way reset substring formed
        continue;
    }

    if (!(a >= 'a' && a <= 'z') &&
        !(charPrevious >= 'a' && charPrevious <= 'z')) {                            // if its a dash or/and apostrophe or/and a not allowed character back to back;
        if (charPrevious == '\'' || charPrevious == '-') intSubstring -= 1;         // dont consider last non alphabet (but valid) as part of the word (the word stands for itself)
        if (intSubstring > 1) intWordCount++;                               // +1 in word count if substring length reached
        intSubstring = 0;                                                   // either way reset substring formed
    }

    intSubstring++; // if no terminating conditions apply, just add a it to substring length count
    charPrevious = a;
}

if (intSubstring > 1 || isSpecialWord(charPrevious)) intWordCount++; // check last word before termination

return intWordCount;
```

- **charCount**
  Return Type – int
  Parameter/s – none
  Description – using the *strInput* field of the class, characters are defined as NOT whitespace, hence the spaces in the string, if there are any occurence, are removed, and the length is simply returned that determines the amount of characters in the said string.

Code Snippet:

```
public int charCount() {
    Utils.isEmpty(strInput);

    String strNoSpace = Utils.rmSpace(strInput);     // remove space
    return strNoSpace.length();                      // return the length
}
```

- **substringCount**
  Return Type - int
  Parameter/s – none
  Description – using the *strInput* field of the class and the argument *strToCompare,* the program builds a pattern using the program-defined method substringPattern(), and then using .matcher(*strInput*) which performs the search, then .results() which returns the stream of raw results, finally .count() counts the elements returned by the stream. The result of this method is casted into an integer and then returned.
  Code Snippet:

```
public int substringCount(String strToCompare) {
    Utils.isEmpty(strInput);

    return (int) substringPattern(strToCompare)     // generate RegEx pattern for substrings
            .matcher(strInput)                      // get matches
            .results()                              // return as results in set and count
            .count();
}
```

- **toLower**
  Return Type – String
  Parameter/s – none
  Description – using the strInput field of the class, this method converts all alphabetic characters within the string into their lowercase equivalents. It makes use of toLowerCase conversion. The structure, spacing, and non-alphabetical symbols remain unmodified. This results in the input string in lowercase.
  Code Snippet:

```
public String toLower() {
    Utils.isEmpty(strInput);
    return strInput.toLowerCase();
}
```

- **toUpper**
  Return Type – String
  Parameter/s – none
  Description – similar to toLower(), but instead transforms the entire strInput into uppercase form. Every alphabetic character is converted, while whitespace and other stuff remain the same. This results in the input string in uppercase.
  Code Snippet:

```
public String toUpper() {
    Utils.isEmpty(strInput);
    return strInput.toUpperCase();
}
```

- **removeVowel**
  Return Type – String
  Parameter/s – none
  Description – utilizing the precompiled ISVOWEL pattern, the method scans strInput and removes all vowel characters. Any occurrence of letters matching the vowel set is stripped out entirely. After removal, a trim() is applied to clear any leading or trailing whitespace. The returned string therefore contains only non-vowel characters of the original input.
  Code Snippet:

```
public String removeVowel() {
    Utils.isEmpty(strInput);
    return ISVOWEL.matcher(strInput).replaceAll(replacement: "").trim();
}
```

- **removeConsonants**
  Return Type – String
  Parameter/s – none
  Description – making use of the precompiled ISCONSONANT pattern, the method eliminates all consonant characters from the strInput. As the removal is applied through pattern replacement, only vowel characters, symbols, and whitespace remain. A final trim() makes sure whitespace is cleaned before returning the resulting no-consonant string.
  Code Snippet:

```
public String removeConsonants() {
    Utils.isEmpty(strInput);
    return ISCONSONANT.matcher(strInput).replaceAll(replacement: "").trim();
}
```

- **isAllowed**
  Return Type – boolean
  Parameter/s – char c
  Description - determines whether a character is permitted in a processed word. This method checks if the input character is a lowercase alphabet letter (a–z), a hyphen (-), or an apostrophe (').
  It returns true if the character belongs to this allowed set; otherwise, it returns false.
  Code Snippet:

```
private static boolean isAllowed(char c) {
return switch (c) {
    case 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j',
         'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't',
         'u', 'v', 'w', 'x', 'y', 'z',
         '-', '\'' -> true;
    default -> false;
    };
}
```

- **isSpecialWord**
  Return Type – boolean
  Parameter – char c
  Description - Checks if the given character represents a special single-letter word. This method identifies characters that are considered standalone words in English, specifically 'a', 'i'. It returns true if the character matches one of these; otherwise, it returns false.
  Code Snippet:

```
private static boolean isSpecialWord(char c) {
return switch (c) {
    case 'i', 'a' -> true;
    default -> false;
    };
}
```

- **acceptStringInput**
  Return Type – String
  Parameter/s – Scanner *scanner*
  Description – this method handles raw string input from the user. It displays a text prompt and collects the entire line typed by the user through the provided Scanner instance. The collected string is then returned.
  Code Snippet:

```
public String acceptStringInput(Scanner scanner) {
    System.out.print(s: " Input Text > ");
    String strTemp = scanner.nextLine();
    System.out.println();
    return strTemp;
}
```

- **acceptChoice**
  Return Type – int
  Parameter/s – *Scanner scanner, int intMaxChoice*
  Description – this method is responsible for safely accepting numerical input that corresponds to a menu selection. It prompts the user for a choice and attempts to read an integer using nextInt(). If invalid input (such as letters or symbols) is entered, it catches InputMismatchException and returns –1. It also validates that the allowed menu range (1 up to intMaxChoice), returning –1 for out-of-bounds values. The finally block clears the scanner buffer.

Code Snippet:

```
try {
    System.out.print(s: " Input Choice > ");
    intChoice = scanner.nextInt();
} catch (InputMismatchException e) {
    System.out.println(" ERROR : " + e);
    return -1;
} finally {
    scanner.nextLine();
}

if (intChoice < 1 || intChoice > intMaxChoice) {
    System.out.println(x: " ERROR : Invalid choice");
    return -1;
}
System.out.println();
return intChoice;
```

- **printHeader**
  Return Type – void
  Parameter/s – none
  Description – this method prints the static header used at the top of the menu interface.
  Code Snippet:

```
public void printHeader() {
    System.out.println(x: "╔═══════════════════════════════════╗");
    System.out.println(x: "║          STRING UTILITY MENU          ╖");
    System.out.println(x: "╚═══════════════════════════════════╝");
}
```

- **printChoices**
  Return Type – void
  Parameter/s – none
  Description – printing the full list of available string utility operations, this method iterates over the arrMenu 2-D array, which contains pairs of method names and corresponding descriptions. For each entry, it prints the method number, method name (with parentheses added for readability), and a brief explanation of the function.
  Code Snippet:

```
public void printChoices(){
    System.out.println(x: " Choose one of the following (input method number)");
    for (int i = 0; i < arrMenu.length; i++) {
        String strMethodName = arrMenu[i][0];
        String strDescription = arrMenu[i][1];
        System.out.printf(format: "%2d) %-20s - %s%n", i + 1, strMethodName + "()", strDescription);
    }
}
```

- **UI**
  Description – upon creation of a UI instance, the constructor initializes the arrMenu two-dimensional string array. Each row consists of a method identifier paired with its description.

Code Snippet:

```
UI() {
    arrMenu = new String[][] {
        {"stringReverse", "Reverse a string"},
        {"isPalindrome", "Check if palindrome"},
        {"isAnagram", "Check if two strings are anagrams"},
        {"wordCount", "Count words in string"},
        {"charCount", "Count characters (with/without spaces)"},
        {"substringCount", "Count occurrences of substring"},
        {"toLower", "Convert to lowercase"},
        {"toUpper", "Convert to uppercase"},
        {"removeVowel", "Remove all vowels"},
        {"removeConsonant", "Remove all consonants"},
        {"exit", "exit program"}
    };
}
```

- **isSecondaryInput**
  Return Type – boolean
  Parameter/s – Scanner scanner, int intMethodName, UI ui
  Description – this method handles the decision-making process for obtaining a secondary input string when a selected string utility method requires one (such as anagram checking or substring counting). It informs the user which method requires a second input, then presents a simple two-option prompt: either input the secondary string or return. Once a valid choice is confirmed through *ui.acceptChoice*, the method returns true only when the user chooses to enter the secondary input; otherwise, false is returned to indicate cancellation which sends user back to main menu.

  Code Snippet:

```
System.out.println(" " + strMethodName[intMethodName - 1]+" needs a secondary string input to compare with");

while (true) {
    System.out.println(x: " 1) input secondary string");
    System.out.println(x: " 2) return");

    intChoice = ui.acceptChoice(scanner, intMaxChoice: 2);

    if (intChoice != -1) break;
}

if (intChoice == 1) {
    System.out.println(x: " Input secondary string input: ");
    return true;
}
return false;
```

- **isPrimaryInput**
  Return Type – boolean
  Parameter/s – Scanner scanner, int intMethodName, UI ui, StringMethods stringProcessor
  Description – this method validates and acquires the *primary* string needed for the selected operation. It first informs the user that the chosen utility method requires a main input string, then gives three possible actions: (1) provide a new string, (2) reuse the existing stored string, or (3) return to main menu. If successful, the input is stored into the stringProcessor object, and the method returns true to proceed

with the selected string manipulation method. Selecting return or failing checks results in false, signaling that the main program should skip execution for this cycle.

Code Snippet:

```
switch (intChoice) {
    case 1 -> {
        System.out.println(x: " Input new string to object: ");
        String strTemp = "";
        while (true) {
            strTemp = ui.acceptStringInput(scanner);
            if (!sentenceCheck(intMinSentence: 3, strTemp)) {
                System.out.println(x: " Requirement : Please input a minimum of 3 sentences");
            }
            else break;
        }
        stringProcessor.setString(strTemp);
        System.out.println(x: " / Using new input string; proceeding with algorithm.");
        pause(scanner);
        return true;
    }
    case 2 -> {
        if (stringProcessor.getString() != null) {
            System.out.println(x: " / Using current/previous input string; proceeding with algorithm.");
            pause(scanner);
            return true;
        } else {
            System.out.println(x: " / Input string field is currently null; set string input first.");
            pause(scanner);
        }
    }
    case 3 -> {
        return false;
```

- **printResult**

  Return Type – void

  Parameter/s – String strResult, String strSecondary, int intChoice, StringMethods stringProcessor

  Description – this formatted output method displays the result of the previous string manipulation method. It shows the primary input currently stored in stringProcessor, then prints according to the nature of the method(e.g it prints wether true or false in isPalindrome, it prints the count in substringCount). The formatting makes a clean and uniform output.

  Code Snippet:

```
switch (intChoice) {
    case 2 -> {
        if (strResult.equalsIgnoreCase(anotherString: "true")) {
            System.out.println(" Result: " + stringProcessor.getString() + " IS A palindrome(" + strResult + ")");
        } else {
            System.out.println(" Result: " + stringProcessor.getString() + " IS NOT A palindrome(" + strResult + ")");
        }
    }
    case 3 -> {
        if (strResult.equalsIgnoreCase(anotherString: "true")) {
            System.out.println(" Result: " + strSecondary + " IS AN anagram(" + strResult + ") of " + stringProcessor.getString());
        } else {
            System.out.println(" Result: " + strSecondary + " IS NOT AN anagram(" + strResult + ") of " + stringProcessor.getString());
        }
    }
    case 4 -> {
        System.out.println(" Result: [" + strResult+ "] words in the input string");
    }
    case 5 -> {
        System.out.println(" Result: [" + strResult + "] characters in the input string");
    }
    case 6 -> {
        System.out.println(" Result: [" + strResult + "] instances of substring (" + strSecondary + ")");
    }
    default -> {
        System.out.println(" Result: " + strResult);
```

- **outro**

  Return Type – void

  Parameter/s – none

Description – the outro for the program that prints the developer's name and a thank you note
Code Snippet:

```
public static void outro() {
    System.out.println(x: " Thank you for using StringMethods");
    System.out.println(x: " by: Villeza, Sean Russell B., CS 2-5");

}
```

- **pause**
  Return Type – void
  Parameter/s – Scanner scanner
  Description – this method temporarily pauses program execution by prompting the user to press the Enter key (any key really, then new line) in order to simulate a real pause in operation that helps user digest content especially when error inputs come.
  Code Snippet:

```
public static void pause(Scanner scanner) {
    System.out.print(s: " press enter to continue...");
    scanner.nextLine();
    System.out.println();
}
```

- **main**
  Return Type – void
  Parameter/s – String[] args
  Description – this is the central control loop of the program, responsible for initializing the scanner, UI handler, and the StringMethods processor object. It displays the menu, validates user input, calls isPrimaryInput and isSecondaryInput when required, executes the corresponding string utility method based on the selected menu index through a switch block, and prints the appropriate results using printResult. It runs inside a loop that continues until the user chooses the exit option. All errors generated by invalid input or argument issues are safely caught and displayed before the loop resumes.

Code Snippet:

```java
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    StringMethods stringProcessor = new StringMethods();

    UI ui = new UI();
    int intChoice;
    boolean boolIsExit = false;

    while (!boolIsExit) {

        ui.printHeader();
        ui.printChoices();
        intChoice = ui.acceptChoice(scanner, intMaxChoice: 11);

        if (intChoice == 11) {
            boolIsExit = true;
            continue; // dont evaluate switch case
        } else if (intChoice == -1) {
            pause(scanner);
            continue;
        }  // dont evaluate switch case

        if(!isPrimaryInput(scanner, intChoice, ui, stringProcessor)) continue; // dont switch case
        String strSecondary;
        try {
            switch (intChoice) {
            case 1 -> printResult(stringProcessor.stringReverse(), strSecondary: "", intChoice, stringProcessor);
            case 2 -> printResult(String.valueOf(stringProcessor.isPalindrome()), strSecondary: "", intChoice, stringProcessor);
            case 3 -> {
                if (isSecondaryInput(scanner, intChoice, ui)) {
                    while (true) {
                        strSecondary = ui.acceptStringInput(scanner);
                        if (!sentenceCheck(intMinSentence: 3, strSecondary)) {
                            System.out.println(x: " Requirement : Please input a minimum of 3 sentences");
                        }
                        else break;
                    }
                    boolean boolTemp = stringProcessor.isAnagram(strSecondary);
                    printResult(String.valueOf(boolTemp), strSecondary, intChoice, stringProcessor);
                } else {
                    break;
                }
            }
            case 4 -> printResult(String.valueOf(stringProcessor.wordCount()),strSecondary: "", intChoice, stringProcessor);
            case 5 -> printResult(String.valueOf(stringProcessor.charCount()), strSecondary: "", intChoice, stringProcessor);
            case 6 -> {
                if (isSecondaryInput(scanner, intChoice, ui)) {
                    strSecondary = ui.acceptStringInput(scanner);
                    int intTemp = stringProcessor.substringCount(strSecondary);
                    printResult(String.valueOf(intTemp), strSecondary, intChoice, stringProcessor);
                } else {
                    break;
                }
            }
            case 7 -> printResult(stringProcessor.makeLowerCase(), strSecondary: "", intChoice, stringProcessor);
            case 8 -> printResult(stringProcessor.makeUpperCase(), strSecondary: "", intChoice, stringProcessor);
            case 9 -> printResult(stringProcessor.removeVowel(), strSecondary: "", intChoice, stringProcessor);
            case 10 -> printResult(stringProcessor.removeConsonants(), strSecondary: "", intChoice, stringProcessor);
            }
        } catch (IllegalArgumentException e) {
```

Developer Notes:

1) **Words** in this program are defined as having alphabet characters and (‘) and (–) only, the only 1 character words are i and a.

```java
public int wordCount() { // words here are defined as substrings > 1 consisting of alphabets. only 2 words are 1 character long, i and a, and "'", "-" are allowed to continue substrings
    Utils.isEmpty(strInput);
```

2) Words are **not their own anagrams**, that is, if a string to compare is exactly equal as the string field of the StringMethod objects it dismisses it immediately as false (not an Anagram)

```java
if(strInput.equalsIgnoreCase(strToCompare)) return false; // a word is not an anagram of itself, it must be a permutation
```

3) Whitespaces are **not** considered as characters in this program in charCount

```
String strNoSpace = Utils.rmSpace(strInput);      // remove space
return strNoSpace.length();                      // return the length
```

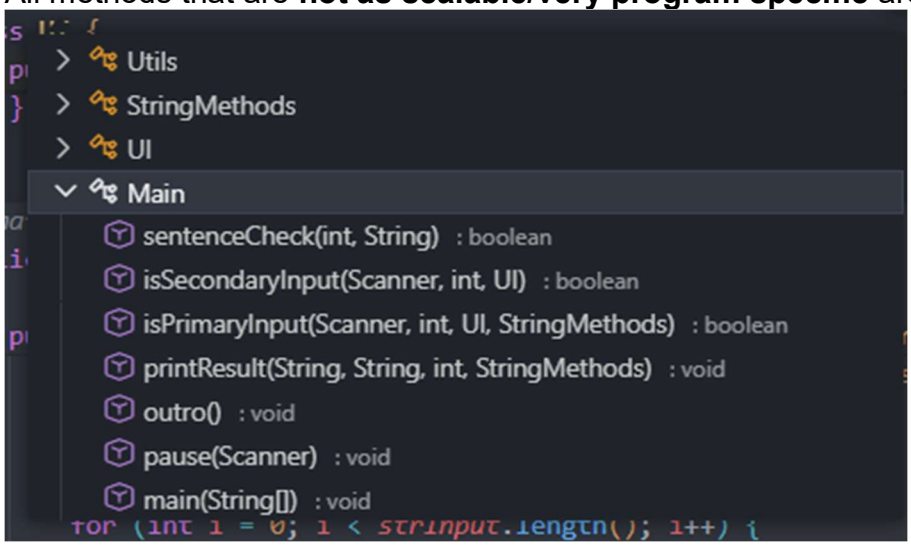
4) We **pre-compile** known patterns that are used a lot to lessen compilation overhead

```
private static final Pattern ISVOWEL =  // compile once pattern for session use
    Pattern.compile(regex: "[aeiou]", Pattern.CASE_INSENSITIVE);

private static final Pattern ISCONSONANT =  // compile once pattern for session use
    Pattern.compile(regex: "[b-df-hj-np-tv-z]", Pattern.CASE_INSENSITIVE);
```

5) All string manipulation methods here run in o(n) time complexity! e.g wordcount;

```
    for(char a: arrText) {   <------ Runs n times

        if (!isAllowed(a)) {                                                    // if its not
            if (intSubstring > 1 || isSpecialWord(charPrevious)) intWordCount++;        // +1 in
            intSubstring = 0;
            charPrevious = a;                                                       // either
            continue;
        }

        if (!(a >= 'a' && a <= 'z') &&
            !(charPrevious >= 'a' && charPrevious <= 'z')) {                            // if
            if (charPrevious == '\'' || charPrevious == '-') intSubstring -= 1;         // do
            if (intSubstring > 1) intWordCount++;                                   // +1 in word
            intSubstring = 0;                                                       // either way
        }

        intSubstring++; // if no terminating conditions apply, just add a it to substring length
        charPrevious = a;
    }

    if (intSubstring > 1 || isSpecialWord(charPrevious)) intWordCount++; // check last word befor

    return intWordCount;
}
```

6) All methods that are **not as scalable/very program specific** are put in main



7) All sample inputs below and in the video are 3 sentences, when applicable.

8) All string inputs are a minimum of **3 sentences except for secondary string of substringCounter**, which allows a minimum of NOT empty, since it is only meant to count the occurrence of some substring(conceptually given to be smaller than original)

```
case 3 -> {
    if (isSecondaryInput(scanner, intChoice, ui)) {
        while (true) {
            strSecondary = ui.acceptStringInput(scanner);
            if (!sentenceCheck(intMinSentence: 3, strSecondary)) {
                System.out.println(x: " Requirement : Please input a minimum of 3 sentences");
            }
            else break;
        }
        boolean boolTemp = stringProcessor.isAnagram(strSecondary);
        printResult(String.valueOf(boolTemp), strSecondary, intChoice, stringProcessor);
    } else {
        break;
    }
}
case 4 -> printResult(String.valueOf(stringProcessor.wordCount()),strSecondary: "", intChoice, stringProcessor);
case 5 -> printResult(String.valueOf(stringProcessor.charCount()), strSecondary: "", intChoice, stringProcessor);
case 6 -> {
    if (isSecondaryInput(scanner, intChoice, ui)) {
        strSecondary = ui.acceptStringInput(scanner);
        int intTemp = stringProcessor.substringCount(strSecondary);
        printResult(String.valueOf(intTemp), strSecondary, intChoice, stringProcessor);
    } else {
        break;
    }
```

9) 11 is the code for exit, from the main menu

```
11) exit()                    - exit program
 Input Choice >
```

10) Sample inputs;

a. **stringReverse()** – Using menu input “1”, and set a new string with 3 sentences, we get the result of the entire string with all characters in reverse order.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()        - Reverse a string
 2) isPalindrome()         - Check if palindrome
 3) isAnagram()            - Check if two strings are anagrams
 4) wordCount()            - Count words in string
 5) charCount()            - Count characters (without spaces)
 6) substringCount()       - Count occurrences of substring
 7) toLower()              - Convert to lowercase
 8) toUpper()              - Convert to uppercase
 9) removeVowel()          - Remove all vowels
10) removeConsonant()      - Remove all consonants
11) exit()                 - exit program
 Input Choice > 1

 Choose an option ; stringReverse needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy
dog jumps today.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 ═══ METHOD RESULT ═══
 Input: The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy dog ju
mps today.
 Result: .yadot spmuj god yzal eht xof kciuq nworb eht revo ylwolS .xof nworb kciuq eht revo ylwols spmuj god yzal eht yadoT .yadot ylwols god yzal eht revo spmuj xof nwor
b kciuq eht
 press enter to continue...
```

b. **isPalindrome()** - Using menu input “2”, a new string is set, isPalindrome identifies this string as not a palindrome.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()        - Reverse a string
 2) isPalindrome()         - Check if palindrome
 3) isAnagram()            - Check if two strings are anagrams
 4) wordCount()            - Count words in string
 5) charCount()            - Count characters (without spaces)
 6) substringCount()       - Count occurrences of substring
 7) toLower()              - Convert to lowercase
 8) toUpper()              - Convert to uppercase
 9) removeVowel()          - Remove all vowels
10) removeConsonant()      - Remove all consonants
11) exit()                 - exit program
 Input Choice > 2

 Choose an option ; isPalindrome needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the la
zy dog jumps today.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 ═══ METHOD RESULT ═══
 Input: The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy dog
 jumps today.
 Result: The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy do
g jumps today. IS NOT A palindrome(false)
 press enter to continue...
```

c. **isAnagram()** – Using menu input “3”, we set a new string as primary input, and another string, the secondary string, to be compared to the primary input. IsAnagram outputs both strings and say they are indeed Anagrams of each other

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()        - Reverse a string
 2) isPalindrome()         - Check if palindrome
 3) isAnagram()            - Check if two strings are anagrams
 4) wordCount()            - Count words in string
 5) charCount()            - Count characters (without spaces)
 6) substringCount()       - Count occurrences of substring
 7) toLower()              - Convert to lowercase
 8) toUpper()              - Convert to uppercase
 9) removeVowel()          - Remove all vowels
10) removeConsonant()      - Remove all consonants
11) exit()                 - exit program
 Input Choice > 3

 Choose an option ; isAnagram needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 isAnagram needs a secondary string input to compare with
 1) input secondary string
 2) return
 Input Choice > 1

 Input secondary string input:
 Input Text > The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy dog jumps today.

 ═══ METHOD RESULT ═══
 Input: The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.
 Result: The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy dog jumps today. IS AN anagram(
true) of The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.
 press enter to continue...
```

d. **wordCount()** – Using menu input “4”, we set a the same string from the previous ones to count the amount of words. wordcount() outputs the total number of words in the input string (33). A screenshot from a website that implements the same concept is added for comparison.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > 4

 Choose an option ; wordCount needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today th
e lazy dog jumps slowly.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 ======= METHOD RESULT =======
 Input: The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy
 dog jumps slowly.
 Result: [33] words in the input string
 press enter to continue...
```

e. **charCount()** – Using menu input “5”, we set the same string, and the amount of characters excluding whitespaces are counted and outputted (141). Again, a screenshot from a website that does the same concept is added for comparison

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > 5

 Choose an option ; charCount needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today th
e lazy dog jumps slowly.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 ======= METHOD RESULT =======
 Input: The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy
 dog jumps slowly.
 Result: [141] characters in the input string
 press enter to continue...
```

f. **substringCount()** – Using menu input “6”, we set a new string that like the previous ones, is 3 sentences long. However, we are prompted to input a secondary string that acts as the specified substring to look for, we input a word for this “brown” for instance, which outputs (3). Similarly, another example is here to supplement that 3 sentences are *also available* to be a substring, which evaluates to (1), because the substring to look for is the same as the input string.

```
Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > 6

 Choose an option ; substringCount needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 substringCount needs a secondary string input to compare with
 1) input secondary string
 2) return
 Input Choice > 1

 Input secondary string input:
 Input Text > brown

 ======= METHOD RESULT =======
 Input: The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.
 Result: [3] instances of substring (brown)
 press enter to continue...
```

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > 6

 Input Choice > 1

 Input new string to object:
 Input Text > The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 substringCount needs a secondary string input to compare with
 1) input secondary string
 2) return
 Input Choice > 1

 Input secondary string input:
 Input Text > The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.

 ======= METHOD RESULT =======
 Input: The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.
 Result: [1] instances of substring (The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jum
 press enter to continue...
```

g. **toLower()** – Using menu input “7”, we set a new string, which outputs a similar string but all uppercased letters in the input is lowercased in the result.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > 7

 Choose an option ; toLower needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy dog jumps today.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 ======= METHOD RESULT =======
 Input: The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy dog jumps today.
 Result: the quick brown fox jumps over the lazy dog slowly today. today the lazy dog jumps slowly over the quick brown fox. slowly over the brown quick fox the lazy dog jumps today.
 press enter to continue...
```

h. **toUpper()** – Using menu input “8”, we set a new string input, and toUpper() outputs the same string but every character is uppercased.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > 8

 Choose an option ; toUpper needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy dog jumps today.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 ======= METHOD RESULT =======
 Input: The quick brown fox jumps over the lazy dog slowly today. Today the lazy dog jumps slowly over the quick brown fox. Slowly over the brown quick fox the lazy dog jumps today.
 Result: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG SLOWLY TODAY. TODAY THE LAZY DOG JUMPS SLOWLY OVER THE QUICK BROWN FOX. SLOWLY OVER THE BROWN QUICK FOX THE LAZY DOG JUMPS TODAY.
 press enter to continue...
```

i. **removeVowel()** – Using menu input “9”, we set a new string input, which outputs a similar string but with all vowels removed and the rest are unaltered (including whitespaces).

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > 9

 Choose an option ; removeVowel needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 ======= METHOD RESULT =======
 Input: The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.
 Result: Th lzy dg tdy jmps slwly vr th qck brwn fx. Brwn fx jmps qck vr th dg tdy th lzy slwly. vr th fx brwn qck tdy th lzy dg jmps slwly.
 press enter to continue...
```

j. **removeConsonants()** – Using menu input “10”, we set a new string again, and outputs a similar string but with all consonant characters removed. All other characters and whitespace is left unaltered.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()       - Reverse a string
 2) isPalindrome()        - Check if palindrome
 3) isAnagram()           - Check if two strings are anagrams
 4) wordCount()           - Count words in string
 5) charCount()           - Count characters (without spaces)
 6) substringCount()      - Count occurrences of substring
 7) toLower()             - Convert to lowercase
 8) toUpper()             - Convert to uppercase
 9) removeVowel()         - Remove all vowels
10) removeConsonant()     - Remove all consonants
11) exit()                - exit program
 Input Choice > 10

 Choose an option ; removeConsonant needs a primary string input
 1) set new string input
 2) use current string input
 3) return
 Input Choice > 1

 Input new string to object:
 Input Text > The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.

 / Using new input string; proceeding with algorithm.
 press enter to continue...

 ======= METHOD RESULT =======
 Input: The lazy dog today jumps slowly over the quick brown fox. Brown fox jumps quick over the dog today the lazy slowly. Over the fox brown quick today the lazy dog jumps slowly.
 Result: e a o oa u o oe e ui o o. o o u ui oe e o oa e a o. Oe e o o ui oa e a o u o.
 press enter to continue...
```

k. **exit** – using menu input “11”, we force the program to exit whenever we want

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()       - Reverse a string
 2) isPalindrome()        - Check if palindrome
 3) isAnagram()           - Check if two strings are anagrams
 4) wordCount()           - Count words in string
 5) charCount()           - Count characters (without spaces)
 6) substringCount()      - Count occurrences of substring
 7) toLower()             - Convert to lowercase
 8) toUpper()             - Convert to uppercase
 9) removeVowel()         - Remove all vowels
10) removeConsonant()     - Remove all consonants
11) exit()                - exit program
 Input Choice > 11

 Thank you for using StringMethods
 by: Villeza, Sean Russell B., CS 2-5
```

l. **input mismatch type** – We try to see if it will not accept an improper input, especially one that asks for an integer in the range of 1-10, for this instance we input the non-numerical character “h”, and see that it catches the exception, prints it, and repeats the process after so, displaying that exceptions are properly handled and does not cause the program to crash.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > h
 ERROR : java.util.InputMismatchException
 press enter to continue...

STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice >
```

m. **out of bounds input** – Similarly we see by inputting a choice out of defined range returns an invalid choice, the program continues after pressing enter, it is pausing so the user can properly see what error has occurred.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
 Input Choice > 56
 ERROR : Invalid choice
 press enter to continue...
```

n. **illegal arguments -** For illegal arguments, the IllegalArgumentException is also printed for user information. The program is then also paused and continues to ask for a new prompt.

```
STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
Input Choice > 1

Choose an option ; stringReverse needs a primary string input
1) set new string input
2) use current string input
3) return
Input Choice > 1

Input new string to object:
Input Text >

/ Using new input string; proceeding with algorithm.
press enter to continue...

ERROR : java.lang.IllegalArgumentException: Input string cannot be empty.
press enter to continue...

STRING UTILITY MENU

Choose one of the following (input method number)
 1) stringReverse()      - Reverse a string
 2) isPalindrome()       - Check if palindrome
 3) isAnagram()          - Check if two strings are anagrams
 4) wordCount()          - Count words in string
 5) charCount()          - Count characters (without spaces)
 6) substringCount()     - Count occurrences of substring
 7) toLower()            - Convert to lowercase
 8) toUpper()            - Convert to uppercase
 9) removeVowel()        - Remove all vowels
10) removeConsonant()    - Remove all consonants
11) exit()               - exit program
Input Choice >
```

Learning Outcomes

This String Manipulation Toolkit project reminded me of the significance of thinking. Developers become inefficient not only because of skill gaps but because they skip planning. Flowcharts, algorithms, and defining the structure of the program. The project's ambiguity was both a blessing and a curse, and many of the decisions I made for myself as a dev is stated in developers notes. I'm glad I didn't rush straight into coding and instead allowed moments during those train rides and walk homes to decide which methods fit together and what responsibilities each one should hold. I learned a lot about Java's string manipulation capabilities. Built-in methods to pattern compilation, comparisons, and parsing techniques that show the inherent properties and advantage of strings. Even though explicit OOP was not required, I naturally applied encapsulation by making the toolkit instantiable and string stored as a field, using getters and setters so the value could be reused without repeatedly asking the user to re-enter it. I also gained experience handling errors properly through try-catch blocks and by throwing custom exceptions for illegal arguments.

The *main struggle* I encountered was dealing with the openness of the task; too much freedom can be overwhelming, and deciding how each method should behave, interact, and remain organized without taking too much code space. Managing user input safely and preventing unexpected crashes also required care on where to declare those structures.

This type of project is *best implemented* in applications where text processing is essential. Such as search, input validation, data cleaning, form parsing, or any backend service that frequently manipulates or filters string data. A modular string toolkit becomes a practical and reusable component in systems that rely heavily on processing, organizing, or transforming textual information.